

Coding Contribution Statement

Factory Inventory & Production Management System (MVP)

Role Name: NBUUB Balalla (Project manager)

Assigned Coding Responsibility: User Authentication

Relevant frontend work

- Login screen implementation in `client/src/pages/Login.tsx`, including form handling, validation messages, loading state, and sign-in submission.
- Authentication state management in `client/src/app/AuthContext.tsx`, including Firebase sign-in, token retrieval, logout flow, refresh of the application user, and user-not-found handling.
- Route protection in `client/src/app/App.tsx` through the `PrivateRoute` wrapper that blocks unauthenticated access and enforces role-aware navigation.
- Authentication request behavior in `client/src/api/client.ts`, where 401 responses trigger a client-side logout event.
- Firebase client initialization in `client/src/utils/firebase.ts` to support email/password sign-in and authenticated API calls.

Relevant backend work

- Authentication middleware in `server/src/middleware/auth.ts` for bearer-token validation, Firebase ID token verification, user lookup, and role enforcement.
- Authenticated profile resolution in `server/src/modules/me/me.routes.ts` so the frontend can resolve the signed-in user record, role, and wholesaler linkage after login.
- User access mapping in `server/src/modules/users/users.routes.ts`, especially owner-controlled user creation and role assignment that supports the authentication-to-authorization flow.
- Authorization usage across protected modules through `requireAuth`, `loadUser`, and `requireRole` as the main access-control path connected to login state.

Supporting implementation work

- Handled the practical handoff between Firebase-authenticated accounts and MongoDB user records so a valid login only becomes a system session when the user is registered in the application database.
- Supported validation and error responses around missing authorization headers, invalid tokens, forbidden roles, and unregistered users.

- Ensured the authentication layer works with the current role model of OWNER, EMPLOYEE, and WHOLESALE.

Conclusion

This coding ownership is documented as the authentication-focused contribution to the project, covering the login experience, authenticated user resolution, token-based backend access control, and the role-gated user access flow that connects Firebase sign-in to the application itself.